IEEE Student Branch WebApp

Complete Development Team Handbook

Project Duration: 12 Weeks (3 Months)
Team Size: 3 Developers
Time Commitment: 2-3 hours/week per person
Start Date: Week of November 18, 2025
Target Launch: February 9, 2026

📋 TABLE OF CONTENTS

1. Project Overview
2. Tech Stack & Setup
3. Team Structure & Roles
4. Development Environment Setup
5. Repository Structure
6. Database Schema Complete Guide
7. API Endpoints Reference
8. Month 1: Detailed Tasks
9. Month 2: Detailed Tasks
10. Month 3: Detailed Tasks
11. Git Workflow & Branching
12. Code Standards & Best Practices
13. Testing Guidelines
14. Deployment Guide
15. Troubleshooting & FAQ

🎯 PROJECT OVERVIEW

What We're Building

A comprehensive web application for IEEE Student Branch operations that automates:

Core Features (MUST HAVE - 3 Months)

1. **Authentication System**
   - Email/password registration and login
   - JWT-based session management
   - Password reset functionality
   - Google OAuth (if time permits)
2. **4-Tier Role System**
   - **Students**: View events, register, answer assignments
   - **Members**: + Create events, mark attendance, upload content
   - **Board Members**: + Form creator, user management
   - **ExCom**: + Full access, analytics, season management
3. **Event Management**
   - Create/edit/delete events
   - Event registration system
   - Capacity management
   - Event status tracking (upcoming/completed)
4. **Attendance System**
   - Digital attendance marking (checkboxes)
   - Bulk select/unselect
   - Export attendance as CSV
   - Email attendees directly
   - Attendance statistics
5. **Smart Email System**
   - Bulk email to groups
   - Dynamic filters:
     - By role (student, member, board, excom)
     - By attendance (attended/didn't attend event)

    - By registration status
    - By season
    - Custom combinations
  - Preview recipients before sending
  - Email history tracking

6. **Season Management System**
   - Create academic year seasons (2024-2025, 2025-2026)
   - Set active season
   - Archive old seasons
   - All data scoped to seasons
   - Season switching for ExCom
7. **Form Builder**
   - Drag-and-drop form creator
   - Field types: text, email, number, textarea, select, radio, checkbox, date, file
   - View form submissions
   - Export responses as CSV
8. **File Upload System**
   - Profile pictures
   - Event posters
   - User resumes and certificates
   - Cloudinary integration
9. **Committee Pages** (Simple)
   - Logo, name, description
   - List of members with positions

Post-Launch Features (Months 4-6)

- Blog/news system
- Assignment system
- Advanced analytics dashboard

- QR code attendance
- Certificate generator

Why This Matters

Current Problem: Board members spend 5-8 hours/week on manual tasks:

- Taking attendance on paper
- Manually copying emails to send announcements
- Creating forms from scratch every time
- No way to track historical data across years

Our Solution: Automate these workflows to save 320+ hours per year

---

🛠️ TECH STACK & SETUP

Frontend Stack

```
- React.js v18+
- React Router v6 (routing)
- Axios (API calls)
- Tailwind CSS or Material-UI (styling)
- React Hook Form (form handling)
- React Context API (state management)
```

Backend Stack

```
- Node.js v18+
- Express.js v4+
- MongoDB + Mongoose (database)
- JWT (jsonwebtoken) (authentication)
- bcryptjs (password hashing)
- Nodemailer (email service)
- Multer (file upload handling)
- Cloudinary SDK (file storage)
- Helmet (security)
- CORS (cross-origin)
- express-rate-limit (rate limiting)
```

Development Tools

```
- VS Code (recommended editor)
- MongoDB Compass (database GUI)
- Postman (API testing)
- Git & GitHub (version control)
- Discord (team communication)
```

Hosting & Services (All Free Tier)

```
- Frontend: Vercel (free, auto-deploy)
- Backend: Render or Railway (free 512MB)
- Database: MongoDB Atlas (free 512MB)
- File Storage: Cloudinary (25GB free)
- Email: Gmail SMTP or SendGrid (100 emails/day)
- Domain: ~$12/year (purchased separately)
```

👥 TEAM STRUCTURE & ROLES

Full Stack Lead (You)

Time: 2-3 hours/week
Focus: Architecture, Integration, Complex Features, DevOps

Responsibilities

- ✅ Database schema design
- ✅ Architectural decisions
- ✅ Complex features (email filters, season management)
- ✅ Frontend + Backend integration
- ✅ Code reviews and PR approvals
- ✅ Deployment and DevOps
- ✅ Security audits
- ✅ Unblocking team members
- ✅ Documentation
- ✅ Testing coordination

Weekly Tasks

- Monday: Review GitHub Projects, assign tasks
- Mid-week: Review PRs, answer questions in Discord
- Thursday: Integration testing
- Friday: Merge approved PRs, deploy to staging
- Weekend: Plan next week

Backend Developer

Time: 2-3 hours/week
Focus: APIs, Database, Server Logic

Responsibilities

- ✅ Create all Mongoose models
- ✅ Build all REST API endpoints
- ✅ Authentication & authorization logic
- ✅ Database queries and optimization
- ✅ Email service integration
- ✅ CSV export functionality
- ✅ File upload backend (Cloudinary)
- ✅ API testing with Postman
- ✅ Error handling

Skills You'll Use

- Node.js & Express
- MongoDB & Mongoose
- JWT authentication
- REST API design
- Async/await patterns
- Middleware creation

Weekly Tasks

- Monday: Check assigned tasks
- Tuesday-Thursday: Build APIs, test in Postman
- Friday: Create PR, update API documentation
- Document all endpoints in Postman collection

Frontend Developer

Time: 2-3 hours/week
Focus: UI/UX, Components, Styling

Responsibilities

- ✅ Build all pages and components
- ✅ Responsive design (mobile-first approach)
- ✅ Form handling and validation
- ✅ Connect to backend APIs
- ✅ State management (Context API)
- ✅ File upload UI
- ✅ Loading states and error handling
- ✅ Styling (Tailwind or Material-UI)
- ✅ User experience optimization

Skills You'll Use

- React.js & Hooks
- React Router
- Axios for API calls
- CSS/Tailwind/Material-UI
- Component design
- State management

Weekly Tasks

- Monday: Check assigned tasks, check if APIs are ready

- Tuesday-Thursday: Build UI components
- Friday: Create PR, test on mobile
- Use mock data if APIs aren't ready yet

💻 DEVELOPMENT ENVIRONMENT SETUP

Initial Setup (Everyone - Week 0)

1. Install Required Software

Node.js (v18 or higher):

```bash
# Check if installed
node --version
npm --version

# If not installed, download from:
# https://nodejs.org/
```

Git:

```bash
# Check if installed
git --version

# If not installed, download from:
# https://git-scm.com/
```

VS Code (Recommended):

```
Download from: https://code.visualstudio.com/
```

MongoDB Compass (Optional but helpful):

```
Download from: https://www.mongodb.com/products/compass
```

---

2. Clone Repositories

```bash
# Create project folder
mkdir ieee-webapp
cd ieee-webapp

# Clone backend
git clone https://github.com/your-org/ieee-webapp-backend.git
cd ieee-webapp-backend
npm install

# Clone frontend
cd ..
git clone https://github.com/your-org/ieee-webapp-frontend.git
cd ieee-webapp-frontend
npm install
```

---

3. Environment Variables Setup

Backend (.env file):

```env
```

```
# Server
PORT=5000
NODE_ENV=development

# Database
MONGODB_URI=mongodb://localhost:27017/ieee-webapp
# Or MongoDB Atlas connection string
# MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/ieee-webapp

# JWT
JWT_SECRET=your-super-secret-jwt-key-change-this-in-production
JWT_EXPIRE=7d

# Email (Gmail SMTP)
EMAIL_USER=your-email@gmail.com
EMAIL_PASS=your-app-specific-password
# Get app password: https://myaccount.google.com/apppasswords

# Cloudinary (Week 8)
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

# Frontend URL (for CORS)
FRONTEND_URL=http://localhost:3000
```

Frontend (.env file):

```env
# API URL
REACT_APP_API_URL=http://localhost:5000/api

# Other configs (if needed)
REACT_APP_NAME=IEEE Student Branch
```

---

4. Test Setup

Backend:

```bash
```

```
cd ieee-webapp-backend
npm run dev

# Should see:
# Server running on port 5000
# MongoDB connected
```

Frontend:

```bash
cd ieee-webapp-frontend
npm start

# Should open browser at http://localhost:3000
```

---

5. VS Code Extensions (Recommended)

Install these extensions:

- ES7+ React/Redux/React-Native snippets
- ESLint
- Prettier - Code formatter
- GitLens
- MongoDB for VS Code
- Thunder Client (Postman alternative)

📁 REPOSITORY STRUCTURE

Backend Repository Structure

```
ieee-webapp-backend/
│
├── src/
│   ├── config/
│   │   └── db.js            # MongoDB connection
│   │
```

```
│   ├── models/
│   │   ├── User.js             # User model (Week 1)
│   │   ├── Season.js           # Season model (Week 1)
│   │   ├── Event.js            # Event model (Week 2)
│   │   ├── Attendance.js       # Attendance model (Week 3)
│   │   ├── EmailLog.js         # Email log model (Week 4)
│   │   ├── Form.js             # Form model (Week 9)
│   │   ├── FormSubmission.js   # Form submission model (Week 9)
│   │   └── Committee.js        # Committee model (Week 10)
│   │
│   ├── routes/
│   │   ├── auth.js             # Auth routes (Week 1)
│   │   ├── users.js            # User management routes (Week 7)
│   │   ├── events.js           # Event routes (Week 2)
│   │   ├── attendance.js       # Attendance routes (Week 3)
│   │   ├── emails.js           # Email routes (Week 4, 6)
│   │   ├── seasons.js          # Season routes (Week 5)
│   │   ├── upload.js           # Upload routes (Week 8)
│   │   ├── forms.js            # Form routes (Week 9)
│   │   └── committees.js       # Committee routes (Week 10)
│   │
│   ├── controllers/
│   │   ├── authController.js     # Auth logic
│   │   ├── userController.js     # User management logic
│   │   ├── eventController.js    # Event logic
│   │   ├── attendanceController.js # Attendance logic
│   │   ├── emailController.js    # Email logic
│   │   ├── seasonController.js   # Season logic
│   │   ├── uploadController.js   # Upload logic
│   │   ├── formController.js     # Form logic
│   │   └── committeeController.js # Committee logic
│   │
│   ├── middleware/
│   │   ├── auth.js             # JWT verification
│   │   ├── roleCheck.js        # Role-based access control
│   │   ├── errorHandler.js     # Global error handler
│   │   ├── upload.js           # Multer upload config
│   │   └── rateLimiter.js      # Rate limiting
│   │
│   ├── utils/
│   │   ├── emailService.js     # Nodemailer setup
│   │   ├── cloudinary.js       # Cloudinary config
│   │   ├── csvExport.js        # CSV generation
│   │   ├── tokenGenerator.js   # JWT token generation
```

```
│   │   └── validators.js        # Input validation
│   │
│   └── server.js               # Express app entry point
│
├── .env                        # Environment variables (DON'T COMMIT)
├── .env.example                # Example env file (commit this)
├── .gitignore
├── package.json
└── README.md
```

Frontend Repository Structure

```
ieee-webapp-frontend/
│
├── public/
│   ├── index.html
│   └── favicon.ico
│
├── src/
│   ├── components/
│   │   ├── common/             # Reusable components
│   │   │   ├── Navbar.js
│   │   │   ├── Footer.js
│   │   │   ├── Loading.js
│   │   │   ├── ErrorMessage.js
│   │   │   └── ConfirmDialog.js
│   │   │
│   │   ├── auth/               # Auth components
│   │   │   ├── LoginForm.js
│   │   │   ├── RegisterForm.js
│   │   │   └── PrivateRoute.js
│   │   │
│   │   ├── events/             # Event components
│   │   │   ├── EventCard.js
│   │   │   ├── EventList.js
│   │   │   ├── EventForm.js
│   │   │   └── EventDetail.js
│   │   │
│   │   ├── attendance/         # Attendance components
│   │   │   ├── AttendanceMarker.js
│   │   │   ├── AttendanceList.js
```

```
│   │   │   └── AttendanceStats.js
│   │   │
│   │   ├── email/              # Email components
│   │   │   ├── EmailComposer.js
│   │   │   ├── FilterBuilder.js
│   │   │   └── RecipientPreview.js
│   │   │
│   │   ├── forms/              # Form builder components
│   │   │   ├── FormBuilder.js
│   │   │   ├── FormField.js
│   │   │   ├── FormFill.js
│   │   │   └── FormResponses.js
│   │   │
│   │   ├── upload/             # Upload components
│   │   │   ├── FileUpload.js
│   │   │   ├── ImagePreview.js
│   │   │   └── UploadProgress.js
│   │   │
│   │   └── users/              # User components
│   │       ├── UserProfile.js
│   │       ├── UserList.js
│   │       └── RoleBadge.js
│   │
│   ├── pages/
│   │   ├── auth/
│   │   │   ├── Login.js
│   │   │   └── Register.js
│   │   │
│   │   ├── events/
│   │   │   ├── EventsPage.js
│   │   │   ├── EventDetailPage.js
│   │   │   └── CreateEventPage.js
│   │   │
│   │   ├── attendance/
│   │   │   └── AttendancePage.js
│   │   │
│   │   ├── email/
│   │   │   └── EmailPage.js
│   │   │
│   │   ├── forms/
│   │   │   ├── FormBuilderPage.js
│   │   │   ├── FormFillPage.js
│   │   │   └── FormResponsesPage.js
│   │   │
```

```
│   │   ├── users/
│   │   │   ├── ProfilePage.js
│   │   │   └── UserManagementPage.js
│   │   │
│   │   ├── committees/
│   │   │   ├── CommitteesPage.js
│   │   │   └── CommitteeDetailPage.js
│   │   │
│   │   ├── seasons/
│   │   │   └── SeasonManagementPage.js
│   │   │
│   │   ├── Dashboard.js
│   │   └── NotFound.js
│   │
│   ├── context/
│   │   ├── AuthContext.js       # Global auth state
│   │   └── SeasonContext.js     # Global season state
│   │
│   ├── services/
│   │   ├── api.js               # Axios instance
│   │   ├── authService.js       # Auth API calls
│   │   ├── eventService.js      # Event API calls
│   │   ├── attendanceService.js # Attendance API calls
│   │   ├── emailService.js      # Email API calls
│   │   ├── seasonService.js     # Season API calls
│   │   ├── uploadService.js     # Upload API calls
│   │   ├── formService.js       # Form API calls
│   │   ├── userService.js       # User API calls
│   │   └── committeeService.js  # Committee API calls
│   │
│   ├── utils/
│   │   ├── formatDate.js        # Date formatting
│   │   ├── validators.js        # Form validators
│   │   └── constants.js         # App constants
│   │
│   ├── hooks/
│   │   ├── useAuth.js           # Auth hook
│   │   └── useSeason.js         # Season hook
│   │
│   ├── App.js
│   ├── index.js
│   └── index.css
│
├── .env                         # Environment variables (DON'T COMMIT)
```

```
├── .env.example          # Example env file (commit this)
├── .gitignore
├── package.json
├── tailwind.config.js     # If using Tailwind
└── README.md
```

🗄️ DATABASE SCHEMA COMPLETE GUIDE

Complete Mongoose Models (Copy-Paste Ready)

1. Season Model

```javascript
```

```
// models/Season.js
const mongoose = require('mongoose');

const seasonSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please add season name'],
    unique: true,
    trim: true,
    // Example: "2024-2025", "2025-2026"
  },
  startDate: {
    type: Date,
    required: [true, 'Please add start date'],
  },
  endDate: {
    type: Date,
    required: [true, 'Please add end date'],
  },
  status: {
    type: String,
    enum: ['active', 'archived'],
    default: 'active',
  },
  isActive: {
    type: Boolean,
    default: false,
    // Only ONE season can be active at a time
  },
  description: {
    type: String,
    maxlength: [500, 'Description cannot be more than 500 characters'],
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },
}, {
  timestamps: true
});

// Ensure only one active season
```

```javascript
seasonSchema.pre('save', async function(next) {
  if (this.isActive) {
    await this.constructor.updateMany(
      { _id: { $ne: this._id } },
      { isActive: false }
    );
  }
  next();
});

// Index for faster queries
seasonSchema.index({ isActive: 1 });
seasonSchema.index({ status: 1 });

module.exports = mongoose.model('Season', seasonSchema);
```

---

2. User Model

```javascript
```

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: [true, 'Please add an email'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [
      /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please add a valid email'
    ]
  },
  password: {
    type: String,
    required: function() {
      return !this.googleId;
    },
    minlength: [6, 'Password must be at least 6 characters'],
    select: false, // Don't return password by default
  },
  googleId: {
    type: String,
    sparse: true,
    unique: true,
  },
  name: {
    type: String,
    required: [true, 'Please add a name'],
    trim: true,
  },
  phone: {
    type: String,
    trim: true,
  },
  profileImage: {
    type: String, // Cloudinary URL
    default: null,
  },
```

```
// SEASON-BASED ROLE MANAGEMENT
seasonMemberships: [{
  season: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Season',
    required: true,
  },
  role: {
    type: String,
    enum: ['student', 'member', 'board_member', 'excom'],
    required: true,
  },
  joinedAt: {
    type: Date,
    default: Date.now,
  },
  leftAt: {
    type: Date,
    default: null,
  },
  isActive: {
    type: Boolean,
    default: true,
  }
}],

// Current season data (for quick access)
currentSeason: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Season',
},
currentRole: {
  type: String,
  enum: ['student', 'member', 'board_member', 'excom'],
  default: 'student',
},

// Additional info
university: String,
major: String,
graduationYear: Number,
ieeeId: String,
```

```
  // Documents
  resume: String, // Cloudinary URL
  certificates: [String], // Array of Cloudinary URLs

  isVerified: {
    type: Boolean,
    default: false,
  },
  resetPasswordToken: String,
  resetPasswordExpires: Date,
}, {
  timestamps: true
});

// Hash password before saving
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) {
    next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Compare password method
userSchema.methods.comparePassword = async function(enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

// Generate JWT token
userSchema.methods.getSignedJwtToken = function() {
  return jwt.sign(
    { id: this._id, role: this.currentRole },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRE || '7d' }
  );
};

// Indexes
userSchema.index({ email: 1 });
userSchema.index({ currentSeason: 1, currentRole: 1 });
userSchema.index({ 'seasonMemberships.season': 1 });

module.exports = mongoose.model('User', userSchema);
```

3. Event Model

```javascript
```

```
// models/Event.js
const mongoose = require('mongoose');

const eventSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Please add event title'],
    trim: true,
    maxlength: [100, 'Title cannot be more than 100 characters'],
  },
  description: {
    type: String,
    required: [true, 'Please add event description'],
    maxlength: [2000, 'Description cannot be more than 2000 characters'],
  },
  season: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Season',
    required: [true, 'Event must belong to a season'],
  },
  type: {
    type: String,
    enum: ['workshop', 'seminar', 'competition', 'meeting', 'social', 'other'],
    default: 'workshop',
  },
  date: {
    type: Date,
    required: [true, 'Please add event date'],
  },
  endDate: {
    type: Date,
  },
  location: {
    type: String,
    required: [true, 'Please add event location'],
  },
  maxAttendees: {
    type: Number,
    default: null, // null means unlimited
  },
  registrationDeadline: {
    type: Date,
  },
```

```
  poster: {
    type: String, // Cloudinary URL
  },
  organizers: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  }],
  registeredUsers: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
    },
    registeredAt: {
      type: Date,
      default: Date.now,
    }
  }],
  status: {
    type: String,
    enum: ['upcoming', 'ongoing', 'completed', 'cancelled'],
    default: 'upcoming',
  },
  isPublished: {
    type: Boolean,
    default: false,
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },
}, {
  timestamps: true
});

// Virtual for registration count
eventSchema.virtual('registrationCount').get(function() {
  return this.registeredUsers.length;
});

// Check if event is full
eventSchema.methods.isFull = function() {
  if (!this.maxAttendees) return false;
  return this.registeredUsers.length >= this.maxAttendees;
```

```
};

// Indexes
eventSchema.index({ season: 1, date: -1 });
eventSchema.index({ status: 1 });
eventSchema.index({ isPublished: 1 });

module.exports = mongoose.model('Event', eventSchema);
```

---

4. Attendance Model

```javascript
```

```
// models/Attendance.js
const mongoose = require('mongoose');

const attendanceSchema = new mongoose.Schema({
  event: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event',
    required: [true, 'Please specify event'],
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'Please specify user'],
  },
  season: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Season',
    required: [true, 'Please specify season'],
  },
  markedAt: {
    type: Date,
    default: Date.now,
  },
  markedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'Please specify who marked attendance'],
  },
  method: {
    type: String,
    enum: ['manual', 'qr_code', 'self_checkin'],
    default: 'manual',
  },
  notes: {
    type: String,
    maxlength: [500, 'Notes cannot be more than 500 characters'],
  },
}, {
  timestamps: true
});

// Prevent duplicate attendance records
attendanceSchema.index({ event: 1, user: 1 }, { unique: true });
```

```
attendanceSchema.index({ season: 1, user: 1 });
attendanceSchema.index({ event: 1 });

module.exports = mongoose.model('Attendance', attendanceSchema);
```

---

5. EmailLog Model

```javascript
```

```
// models/EmailLog.js
const mongoose = require('mongoose');

const emailLogSchema = new mongoose.Schema({
  subject: {
    type: String,
    required: [true, 'Please add email subject'],
  },
  body: {
    type: String,
    required: [true, 'Please add email body'],
  },
  season: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Season',
  },

  // Recipients info
  recipientCount: {
    type: Number,
    required: true,
  },
  recipientEmails: [{
    type: String,
  }],

  // Filters used (for advanced email system)
  filters: {
    roles: [String],
    attendedEvents: [mongoose.Schema.Types.ObjectId],
    notAttendedEvents: [mongoose.Schema.Types.ObjectId],
    customList: [String],
  },

  sentBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
  },

  sentAt: {
    type: Date,
    default: Date.now,
```

```
  },

  status: {
    type: String,
    enum: ['scheduled', 'sending', 'sent', 'failed'],
    default: 'sent',
  },

  scheduledFor: Date,

  errorMessage: String,
}, {
  timestamps: true
});

// Indexes
emailLogSchema.index({ sentBy: 1, sentAt: -1 });
emailLogSchema.index({ status: 1 });

module.exports = mongoose.model('EmailLog', emailLogSchema);
```

---

6. Form Model

```javascript
// models/Form.js
const mongoose = require('mongoose');

const formSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Please add form title'],
    trim: true,
  },
  description: {
    type: String,
    maxlength: [1000, 'Description cannot be more than 1000 characters'],
  },
  season: {
```